TrashUQ: Federated Learning for Intelligent Waste Monitoring on Arduino UNO Q

Aleix BERTRAN ANDREU [a] Pol LLINÀS VAQUER [a] Bru PALLÀS VARGUÉS [a]
Aleix ROSINACH OLIVART [a]
[a] *Universitat de Lleida, Lleida, Catalonia*

Abstract. This paper presents TrashUQ, a waste-monitoring platform that integrates Arduino UNO Q edge nodes, MQTT telemetry, and a gRPC Federated Learning (FL) coordinator. We report two validations. First, a real two-device deployment confirms end-to-end telemetry and FL round-trip behavior (282 persisted MQTT records, model version 0→26, 25/30 successful aggregations). Second, a controlled non-IID FL simulation (Dirichlet $\alpha = 0.3$) with 2, 5, 10, and 20 clients shows stable FedAvg convergence over 25 rounds, with final accuracies around 93–94%. We further demonstrate that stochastic rounding gradient quantization achieves 4–16× model compression with accuracy loss below 2.84 p.p. at 20 clients, outperforming Top-k sparsification which either saves no communication or degrades accuracy by 9+ p.p. FL convergence is robust across a wide range of non-IID intensities ($\alpha = 0.1$–1.0). Results indicate that FL orchestration is feasible on this embedded stack, and that stochastic rounding is the preferred method for communication-constrained edge deployments.

Keywords. Federated Learning, Edge AI, Waste Classification, Arduino UNO Q, MQTT, TrashNet

1. Introduction

Waste mismanagement poses critical environmental and public health challenges. Intelligent waste sorting systems powered by deep learning can significantly improve recycling rates, but conventional approaches rely on centralized cloud infrastructure that raises privacy, latency, and bandwidth concerns [1]. Federated Learning (FL) addresses these limitations by training models collaboratively across edge devices while keeping data local [2,3].

Several communication-efficient FL strategies have been proposed. Gradient sparsification (Top-k [4]) keeps the largest-magnitude updates and zeros the rest, but requires transmitting integer indices alongside values, so its byte savings materialize only at very low keep ratios—which degrade accuracy. Stochastic quantization (QSGD [5]) and 1-bit sign compression (SignSGD [6]) compress each update uniformly without index overhead. Unified frameworks (FedCOM [7]) combine both ideas. Hierarchical FL [8,9] reduces coordinator load through multi-level aggregation, but adds topology complexity unsuitable for flat classroom-scale deployments. Despite their theoretical promise, most

evaluations remain purely simulated on high-end hardware, leaving open questions about practical deployment on resource-constrained boards. The Arduino UNO Q—featuring a Qualcomm QRB2210 processor—enables such a deployment, and we adopt stochastic rounding quantization because it introduces no hyperparameter tuning at the edge and integrates directly with TFLite delegates.

This paper presents TrashUQ, a complete FL-based waste monitoring system deployed on two Arduino UNO Q boards. Our contributions are:

1. A real two-node FL deployment with MQTT telemetry validation, confirming end-to-end FL round-trip behavior (282 records, 25/30 successful aggregations).
2. FL simulation results on a controlled synthetic dataset with FedAvg across 2–20 non-IID clients, demonstrating stable convergence around 93–94% accuracy and robustness across $\alpha \in \{0.1, 0.3, 0.5, 1.0\}$.
3. Stochastic rounding gradient quantization evaluated at four precision levels (2, 4, 6, 8 bits), achieving 4–16$\times$ model compression with accuracy loss below 2.84 p.p. at 20 clients, and compared against Top-k sparsification, where stochastic rounding saves 37–44% of communication with zero accuracy loss while Top-k either saves nothing or degrades accuracy by 9+ p.p.
4. Open-source implementation at `https://github.com/TrashUQ/TrashUQ` [10].

2. System Architecture

Each Arduino UNO Q has a Qualcomm QRB2210 processor (Debian Linux) and an STM32U585 microcontroller, connected over Wi-Fi to a shared backend (MQTT broker, PostgreSQL, FastAPI REST API, gRPC FL coordinator). The system uses two disjoint communication planes: MQTT for lightweight telemetry and gRPC for structured FL coordination, preventing contention between streaming payloads and bulk model transfers.

The edge daemon follows capture/classify/act logic using a MobileNetV2 TFLite model (classes: cardboard, glass, paper, plastic). User corrections are stored locally for on-device fine-tuning. The trainable model is a frozen MobileNetV2 backbone with a small learnable head. After accumulating new labels, a node fetches global weights, trains locally, and submits an update via gRPC; the coordinator applies FedAvg [1]. Boards publish status and FL metrics to `arduino/<device-id>/#` via MQTT.

3. Hardware Validation

Two Arduino UNO Q nodes (Debian 13) ran the edge daemon against the shared backend during a $\sim$12-minute session. End-to-end ingestion was validated across Edge $\rightarrow$ MQTT $\rightarrow$ Backend $\rightarrow$ PostgreSQL $\rightarrow$ Frontend, persisting 282 MQTT rows (147 from `unoq-01`, 135 from `unoq-02`).

Both nodes successfully completed FL cycles (`Join`, `GetGlobalModel`, `SubmitUpdate`), reaching model version 26 with 25/30 successful aggregations (83.3%). Five submissions were rejected as stale during concurrent updates, confirming correct monotonic round enforcement. Round submission latency was low (median $\sim$25 ms, max 105 ms). Mean local accuracy was 0.921 (`unoq-01`) and 0.871 (`unoq-02`). Both devices used `-fake-camera` and HTTP label injection to exercise FL deterministically; camera-in-the-loop latency is left for future work.

Table 1. FL simulation summary (mean across 3 seeds).

Clients	Acc. (%)	Loss	Round Time (s)	Comm. (MB)
2	93.75	0.2192	0.005	0.394
5	93.56	0.2004	0.005	0.985
10	93.37	0.2018	0.006	1.970
20	93.75	0.1933	0.008	3.941

Table 2. Quantization results (20 clients, mean across 3 seeds).

Config	Acc. (%)	Loss	Comm. (MB)	Compression
float32	93.75	0.1933	3.941	1.0×
8-bit	93.75	0.1935	2.474	∼4×
4-bit	93.75	0.1976	2.229	∼8×
2-bit	90.91	0.2990	2.229	∼16×

4. FL Simulation and Communication Efficiency

We ran FedAvg for 25 rounds with non-IID partitions (Dirichlet $\alpha = 0.3$), using 3 seeds per setting with client counts {2, 5, 10, 20}. Each run used local epochs = 2, batch size = 16, SGD with learning rate 0.18, and full participation.

To ensure fully reproducible experiments without external dependencies, we use a synthetic dataset rather than the full TrashNet [11] collection (the simulator can optionally load real images when available). The pipeline generates 16×16 grayscale images (4 waste classes, 220 samples/class) from class-specific prototypes blended via convex combination with Gaussian noise, brightness jitter, and pixel roll.

4.1. Convergence and Communication Efficiency

Across all settings, FL converged stably with final accuracy near 93–94% and no failed rounds (Table 1). Mean round duration increased from 0.005 s (2 clients) to 0.008 s (20 clients), while total communication grew approximately linearly from 0.394 MB to 3.941 MB.

4.2. Gradient Quantization and Method Comparison

To address the communication bottleneck, we integrate stochastic rounding quantization into the FedAvg pipeline. After local training, each client quantizes weight updates before transmission; the server dequantizes before averaging. We evaluate four configurations: float32, 8-bit (∼4×), 4-bit with uint8 packing (∼8×), and 2-bit (∼16×).

Both 8-bit and 4-bit preserve accuracy identically (93.75%) with up to 43% communication savings. Even 2-bit quantization (∼16×) achieves 90.91% at 20 clients (2.84 p.p. drop), indicating that higher client counts absorb quantization noise. We further compare against Top-k sparsification [4] (Table 3). Top-50% matches float32 accuracy but saves no communication because transmitting int32 indices offsets the value reduction. At higher compression (Top-10%, Top-1%) accuracy degrades by 9–30 p.p. while savings plateau at 40–49%. Stochastic rounding, in contrast, compresses uniformly

Table 3. Method comparison (20 clients, mean across 3 seeds).

Method	Acc. (%)	Comm. (MB)	Drop (p.p.)
float32	93.75	3.922	—
SR-4bit	93.75	2.210	0.00
SR-8bit	93.75	2.455	0.00
Top-50%	93.56	3.925	0.19
Top-10%	84.66	2.357	9.09
Top-1%	63.83	2.003	29.9

without index overhead, achieving 37–44% savings at identical accuracy. Repeating the baseline across $\alpha \in \{0.1, 0.3, 0.5, 1.0\}$ confirms FedAvg accuracy between 92.80% and 94.51%—a $\sim$1.6 p.p. range.

5. Conclusion and Future Work

TrashUQ validates FL on real Arduino UNO Q hardware (two-node deployment, 282 MQTT records, 25/30 successful FL aggregations) and in controlled simulation (FedAvg, 2–20 non-IID clients, 93–94% accuracy). Stochastic rounding achieves 4–16$\times$ compression with accuracy loss below 2.84 p.p. at 20 clients, outperforming Top-k sparsification which either saves no bytes or degrades accuracy by 9+ p.p. FL convergence is robust across non-IID intensities ($\alpha = 0.1$ to 1.0). By contrasting our flat topology with hierarchical FL and comparing stochastic rounding to sparsification, we position TrashUQ as a practical alternative requiring no hyperparameter tuning at the edge. Code at `https://github.com/TrashUQ/TrashUQ`; future work includes camera-in-the-loop validation and on-device quantized deployment.

Acknowledgements

The authors thank Jordi Planes Cid from the Universitat de Lleida for guidance and support during this work.

References

[1] McMahan HB, Moore E, Ramage D, Hampson S, y Arcas BA. Communication-Efficient Learning of Deep Networks from Decentralized Data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS); 2017. p. 1273-82.

[2] Li T, Sahu AK, Talwalkar A, Smith V. Federated Optimization in Heterogeneous Networks. In: Proceedings of Machine Learning and Systems (MLSys); 2020. .

[3] Kairouz P, McMahan HB, Avent B, et al. Advances and Open Problems in Federated Learning. Foundations and Trends in Machine Learning. 2021;14(1–2):1-210.

[4] Aji AF, Heafield K. Sparse Communication for Distributed Gradient Descent. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP); 2017. p. 440-50.

[5] Alistarh D, Grubic D, Li J, Tomioka R, Vojnovic M. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding. In: Advances in Neural Information Processing Systems (NeurIPS); 2017. p. 1709-20.

[6] Bernstein J, Wang YX, Azizzadenesheli K, Anandkumar A. signSGD: Compressed Optimisation for Non-Convex Problems. In: Proceedings of the 35th International Conference on Machine Learning (ICML). vol. 80. PMLR; 2018. p. 560-9.

[7] Haddadpour F, Kamani MM, Mokhtari A, Mahdavi M. Federated Learning with Compression: Unified Analysis and Sharp Guarantees. IEEE Transactions on Information Theory. 2021;67(7):4750-76.

[8] Liu L, Zhang J, Song SH, Letaief KB. Client-Edge-Cloud Hierarchical Federated Learning. In: IEEE International Conference on Communications (ICC); 2020. p. 1-6.

[9] Abad MSH, Ozfatura E, Gunduz D, Ercetin O. Hierarchical Federated Learning Across Heterogeneous Cellular Networks. In: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP); 2020. p. 8866-70.

[10] Team T. TrashUQ Project Repository; 2026. `https://github.com/TrashUQ/TrashUQ`.

[11] Thung G, Yang M. Classification of Trash for Recyclability Status; 2016. `http://cs229.stanford.edu/proj2016/report/ThungYang-ClassificationOfTrashForRecyclabilityStatus-report.pdf`. Stanford CS229 Project Report.